浙江大学
ZHEJIANG UNIVERSITY

# 面向智能机器人的漏洞挖掘与自动化攻击路径推理

D I S S E R T A T I O N  D E F E N S E


Instructor: 冯博、郑兆烜

Reporter：王文烁

Team mates：王文烁、洪宇童


2025.12.5

# Catalog

CONTENTS

01 分工情况

02 核心工作

03 当前主要问题

04 下阶段工作安排

# 01

## 分工情况

# 01 分工情况

4月，完成机械臂开发环境搭建

6月，完成机械臂全部运动功能复现，并得到其运动特性等数据

8月，调研DoS攻击等，研读相关论文，确定研究思路

11月，仿照MULVAL搭建轻量级软件环境，初步复现推理过程

9月，分析机械臂源码与环境依赖，进行脆弱性分析

# 02

## 核心工作

# 1、MulVAL

**MulVAL是基于Datalog的多主机、多阶段漏洞分析框架，核心目标是：**

· 自动整合漏洞数据库（OVAL/ICAT）、网络配置、主机配置等信息，无需人工干预

· 高效推理网络中潜在的攻击路径，支持数千台主机规模的快速分析

· 检测违反安全政策的违规行为，为防御措施提供精准依据

**MulVAL: A Logic-based Network Security Analyzer** *

Xinming Ou Sudhakar Govindavajhala Andrew W. Appel
*Princeton University*
{*xou, sudhakar, appel*}@*cs.princeton.edu*

| 建模元素 | 论文定义（通用） | 机械臂场景实例 |
|---|---|---|
| 事实（Facts） | 客观存在的漏洞、配置、权限信息 | vulExists(mycobot, 'CVE-2024-5806', moveit)（机械臂漏洞）<br>networkService(mycobot, ros2_comm, TCP, 11311, ubuntu)（ROS2服务配置）<br>hacl(remote_attacker, mycobot, TCP, 11311)（网络访问控制） |
| 规则（Rules） | 描述攻击行为、权限传播的逻辑关系 | controlArm(Attacker, Host) :- escalateToRoot(Attacker, Host), filePath(Host, root, JointCtrlPath)（机械臂控制规则） |
| 政策（Policy） | 明确允许的访问行为，默认禁止所有 | allow(sysAdmin, control, mycobot_joint)（仅管理员可控制关节） |

**自动推理核心流程**

**1. 攻击模拟阶段（推导所有可能攻击路径）**

基于Datalog规则，自动化串联攻击链条

**2. 政策检查阶段（检测违规行为）**

对比攻击模拟结果与安全政策，未明确允许的访问即判定为违规

# 2、XSB配置与推理

**在研究中我们采用了XSB最新版本，并依照论文做如下规则配置。注意我们这里并未对最新漏洞信息和机械臂参数做相应的爬取，仅模仿论文验证推理结果的有效性**

Open mycobot_mulval.xsb ~/XSB/bin

```
malicious(attacker).  % mark malicious subject

% Reference: allow(subject, action, resource)
allow(admin, control, mycobot_joint).  % allow admin to control joints
allow(ubuntu, read, ros2_data).  % allow ubuntu user to read ROS2 data

% ==================== 5. Inference rules (reuse core rules from paper sec.4.1) ====================
% Rule 1: Remote vuln exploitation -> execute arbitrary code (added hasAccount linking subject and source host)
execCode(Attacker, Host, Priv) :-
    hasAccount(Attacker, SrcHost, _),  % binding: attacker -> corresponding source host (remote_attacker)
    vulExists(Host, VulID, Prog),
    vulProperty(VulID, remoteExploit, privEscalation),
    networkService(Host, Prog, Protocol, Port, Priv),
    hacl(SrcHost, Host, Protocol, Port),  % match network rule using bound SrcHost
    malicious(Attacker).

% Rule 2: Unencrypted communication -> obtain authentication info (also uses linking)
obtainAuthInfo(Attacker, Host) :-
    hasAccount(Attacker, SrcHost, _),  % attacker -> source host (remote_attacker)
    vulExists(Host, VulID, Prog),
    vulProperty(VulID, remoteExploit, infoLeak),
    networkService(Host, Prog, TCP, 11311, _),
    hacl(SrcHost, Host, TCP, 11311),  % match network rule
    malicious(Attacker).

% Rule 3: obtain auth info + identity bypass -> escalate to root
escalateToRoot(Attacker, Host) :-
    obtainAuthInfo(Attacker, Host),
    vulExists(Host, 'CVE-2024-5806', moveit),
    networkService(Host, joint_control, TCP, 8080, root).

% Rule 4: root privilege -> control robotic arm (attack goal)
controlArm(Attacker, Host) :-
    (execCode(Attacker, Host, root); escalateToRoot(Attacker, Host)),
    filePath(Host, root, '/opt/ros/humble/lib/joint_state_publisher/joint_state_publisher').

% Rule 5: policy violation detection (paper sec.4.5 format)
policyViolation(Attacker, Action, Resource) :-
    controlArm(Attacker, mycobot),
    Action = control,
    Resource = mycobot_joint,
    not allow(Attacker, control, mycobot_joint).
```

# 运行Demo



**如图所示，可以看到自动攻击路径推理系统相当成功，在这个情境下：攻击者（attacker）通过自身与远程访问主体的账号绑定（hasAccount），利用机械臂的 ROS2 未加密通信漏洞（ROS2-UNENCRYPTED）获取验证信息（obtainAuthInfo），经权限提升（escalateToRoot）成功控制机械臂（controlArm），最终触发“控制机械臂关节（mycobot_joint）”的安全政策违规。**

what@ubuntu: ~/XSB/bin

```
[Compiling ./mycobot_mulval.xsb]
% Specialising partially instantiated calls to vulProperty/3
% Specialising partially instantiated calls to networkService/5
% Specialising partially instantiated calls to hacl/4
% Specialising partially instantiated calls to vulExists/3
% Specialising partially instantiated calls to filePath/3
% Specialising partially instantiated calls to allow/3
[./mycobot_mulval.xsb compiled, cpu time used: 0.031 seconds]
[./mycobot_mulval.xsb loaded]

yes
| ?- controlArm(Attacker, mycobot).

Attacker = attacker

yes
| ?- policyViolation(Attacker, Action, Resource).

Attacker = attacker
Action = control
Resource = mycobot_joint

yes
| ?-
```

what@ubuntu: ~/XSB/bin

```
yes
| ?- spy(policyViolation/3).
Spy point set on usermod:policyViolation/3

yes
[debug]
| ?- policyViolation(Attacker, Action, Resource).
** (0) Call: policyViolation(_h304,_h346,_h388) ?
   (1) Call: controlArm(_h304,mycobot) ?
   (2) Call: execCode(_h304,mycobot,root) ?
   (3) Call: hasAccount(_h304,_h563,_h560) ?
   (3) Exit: hasAccount(attacker,remote_attacker,malicious) ?
   (4) Call: vulExists(mycobot,_h586,_h587) ?
   (4) Exit: vulExists(mycobot,'CVE-2024-5806',moveit) ?
   (5) Call: networkService(mycobot,moveit,_h610,_h611,root) ?
   (5) Fail: networkService(mycobot,moveit,_h610,_h611,root) ?
   (4) Redo: vulExists(mycobot,'CVE-2024-5806',moveit) ?
   (4) Exit: vulExists(mycobot,'ROS2-UNENCRYPTED',ros2_comm) ?
   (4) Redo: vulExists(mycobot,'ROS2-UNENCRYPTED',ros2_comm) ?
   (4) Fail: vulExists(mycobot,_h586,_h587) ?
   (3) Redo: hasAccount(attacker,remote_attacker,malicious) ?
   (3) Exit: hasAccount(admin,mycobot,ubuntu) ?
   (6) Call: vulExists(mycobot,_h586,_h587) ?
   (6) Exit: vulExists(mycobot,'CVE-2024-5806',moveit) ?
   (7) Call: networkService(mycobot,moveit,_h610,_h611,root) ?
   (7) Fail: networkService(mycobot,moveit,_h610,_h611,root) ?
   (6) Redo: vulExists(mycobot,'CVE-2024-5806',moveit) ?
   (6) Exit: vulExists(mycobot,'ROS2-UNENCRYPTED',ros2_comm) ?
   (6) Redo: vulExists(mycobot,'ROS2-UNENCRYPTED',ros2_comm) ?
   (6) Fail: vulExists(mycobot,_h586,_h587) ?
   (3) Redo: hasAccount(admin,mycobot,ubuntu) ?
   (3) Fail: hasAccount(_h304,_h563,_h560) ?
   (2) Fail: execCode(_h304,mycobot,root) ?
   (8) Call: escalateToRoot(_h304,mycobot) ?
   (9) Call: obtainAuthInfo(_h304,mycobot) ?
   (10) Call: hasAccount(_h304,_h575,_h572) ?
   (10) Exit: hasAccount(attacker,remote_attacker,malicious) ?
   (11) Call: vulExists(mycobot,_h598,_h599) ?
   (11) Exit: vulExists(mycobot,'CVE-2024-5806',moveit) ?
   (11) Redo: vulExists(mycobot,'CVE-2024-5806',moveit) ?
   (11) Exit: vulExists(mycobot,'ROS2-UNENCRYPTED',ros2_comm) ?
   (12) Call: malicious(attacker) ?
   (12) Exit: malicious(attacker) ?
   (9) Exit: obtainAuthInfo(attacker,mycobot) ?
   (8) Exit: escalateToRoot(attacker,mycobot) ?
   (1) Exit: controlArm(attacker,mycobot) ?
** (0) Exit: policyViolation(attacker,control,mycobot_joint) ?

Attacker = attacker
Action = control
Resource = mycobot_joint

yes
```

# 脆弱性剖析



依赖漏洞初筛

①使用 pip freeze > requirements.txt 固定当前虚拟环境中的全部依赖版本。

②通过 pip-audit -r requirements.txt 扫描依赖，发现 mcp 1.16.0 命中公开漏洞 CVE-2025-66416

源码静态分析

①对 mycobot_280_arduino 目录下 11 个 Python 文件运行 Semgrep 规则集（p/ci、p/python）。

②使用 Bandit 进行补充检查，未发现命令注入、不安全函数等高危 Python 语言层问题。

依赖修复与交叉验证

① 将 mcp 升级至 1.23.1，重新执行 pip freeze 更新 requirements.txt。

② 使用 safety check -r requirements.txt 和修复后的 pip-audit 复扫，确认依赖集合中0 个已知漏洞。

# 依赖与源码层面的自动化安全检测



```
(venv) kbwhite@KBdeMacBook-Air r
  --no-deps \
  --progress-spinner off \
  --timeout 20 \
  --disable-pip

WARNING:pip_audit._cli:--no-deps
WARNING:pip_audit._cli:Consider
No known vulnerabilities found
```

audit扫描结果图

```
We highly encourage switching to the new `scan` command which is easier to use, more powerful, and can be set

+==============================================================================================================
======+


+==============================================================================================================

                              /$$$$$$            /$$
                             /$$__  $$          | $$
          /$$$$$$$  /$$$$$$ | $$  \__//$$$$$$  /$$$$$$   /$$   /$$
         /$$_____/ |____  $$| $$$$   /$$__  $$|_  $$_/  | $$  | $$
        |  $$$$$$   /$$$$$$$| $$_/  | $$$$$$$$  | $$    | $$  | $$
         \____  $$ /$$__  $$| $$    | $$_____/  | $$ /$$| $$  | $$
         /$$$$$$$/|  $$$$$$$| $$    |  $$$$$$$  |  $$$$/|  $$$$$$$
        |_______/  \_______/|__/     \_______/   \___/   \____  $$
                                                             /$$  | $$
                                                            |  $$$$$$/
  by safetycli.com                                           \______/

+==============================================================================================================

 REPORT

  Safety v3.7.0 is scanning for Vulnerabilities...
  Scanning dependencies in your files:

  -> requirements.txt

  Using open-source vulnerability database
  Found and scanned 100 packages
  Timestamp 2025-12-04 21:57:37
  0 vulnerabilities reported
  0 vulnerabilities ignored
+==============================================================================================================

 No known security vulnerabilities reported.

+==============================================================================================================


+==============================================================================================================
======+
```

safety交叉验证扫描结果

# 依赖与源码层面的自动化安全检测



```
Semgrep CLI

Scanning 11 files (only git-tracked) with 145 Code rules:

  CODE RULES

  Language      Rules   Files          Origin      Rules
  ─────────────────────────────        ──────────────────
  python           19      11          Community     145
  <multilang>       2      11

  SUPPLY CHAIN RULES

  💎 Sign in with `semgrep login` and run
     `semgrep ci` to find dependency vulnerabilities and
     advanced cross-file findings.

  PROGRESS

  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 100% 0:00:00

Scan Summary
✅ Scan completed successfully.
 • Findings: 0 (0 blocking)
 • Rules run: 21
 • Targets scanned: 11
 • Parsed lines: ~100.0%
 • Scan was limited to files tracked by git
 • For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 21 rules on 11 files: 0 findings.
💎 Missed out on 1185 pro rules since you aren't logged in!
⚡ Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.
(need more rules? `semgrep login` for additional free Semgrep Registry rules)
```

使用 Semgrep 通用规则扫描结果图

```
✨ If Semgrep missed a finding, please send us feedback to let us know!
   See https://semgrep.dev/docs/reporting-false-negatives/
(venv) kbwhite@KBdeMacBook-Air mycobot_280 % semgrep --config p/python \
  mycobot_280_arduino/mycobot_280_arduino

Semgrep CLI

Scanning 11 files (only git-tracked) with 151 Code rules:

  CODE RULES
  Scanning 11 files with 151 python rules.

  SUPPLY CHAIN RULES

  💎 Sign in with `semgrep login` and run
     `semgrep ci` to find dependency vulnerabilities and
     advanced cross-file findings.

  PROGRESS

  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 100% 0:00:00

Scan Summary
✅ Scan completed successfully.
 • Findings: 0 (0 blocking)
 • Rules run: 151
 • Targets scanned: 11
 • Parsed lines: ~100.0%
 • Scan was limited to files tracked by git
 • For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 151 rules on 11 files: 0 findings.
💎 Missed out on 637 pro rules since you aren't logged in!
⚡ Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.
(need more rules? `semgrep login` for additional free Semgrep Registry rules)
```

Semgrep Python 规则扫描结果图

# 依赖与源码层面的自动化安全检测



```
(venv) kbwhite@KBdeMacBook-Air mycobot_280 % bandit -r mycobot_280_arduino/mycobot_280_arduino \
  -f json -o bandit_result.json
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[json]  INFO    JSON output written to file: bandit_result.json
```

## 使用 Bandit 进行补充检查

bandit_result.json

Users > kbwhite > Desktop > mycobot_ws > src > mycobot_ros2 > mycobot_280 > {} b


```
1  {
2    "errors": [],
3    "generated_at": "2025-12-04T14:03:35Z",
4    "metrics": {
5      "_totals": {
6        "CONFIDENCE.HIGH": 7,
7        "CONFIDENCE.LOW": 0,
8        "CONFIDENCE.MEDIUM": 17,
9        "CONFIDENCE.UNDEFINED": 0,
10       "SEVERITY.HIGH": 0,
11       "SEVERITY.LOW": 7,
12       "SEVERITY.MEDIUM": 17,
13       "SEVERITY.UNDEFINED": 0,
14       "loc": 1346,
15       "nosec": 0,
16       "skipped_tests": 0
17     },
```


发现中危漏洞17个，低危漏洞7个，主要是一些中低危的编码安全问题。

① B108：hardcoded_tmp_directory（中危）

用了硬编码的 /tmp/... 临时文件路径来做锁。
对应 CWE-377：临时文件 / 目录使用不安全。
在多用户或不可信本地用户场景下，别人可以提前创建同名文件、做符号链接攻击，导致：
竞争条件 / 死锁
或者让程序以为拿到锁，结果读写的是别人准备好的东西。

② B110：try/except/pass（低危）

CWE-703：不正确的错误处理。

# Teleop 控制逻辑的手工审计与安全风险



## 1、控制逻辑手工审计概览

在工具扫描基础上，选取与机械臂运动最相关的几个节点做手工审计：listen_real.py、listen_real_service.py、simple_gui.py、teleop_keyboard.py

结合 Bandit 报告和代码语义，重点关注：

本地锁与临时文件的实现方式

异常处理与错误反馈

键盘遥操作流程中的物理安全风险

目标：找出 自动化工具难以理解、但会影响机械臂运行安全和可维护性的设计问题

mycobot_280_arduino

mycobot_280_arduino

| Name | Date Modified |
|---|---|
| __init__.py | Today at 8:10 PM |
| camera_display.py | Today at 8:10 PM |
| follow_display.py | Today at 8:10 PM |
| listen_real_of_topic.py | Today at 8:10 PM |
| listen_real_service.py | Today at 8:10 PM |
| listen_real.py | Today at 8:10 PM |
| opencv_camera.py | Today at 8:10 PM |
| simple_gui.py | Today at 8:10 PM |
| slider_control_adaptive_gripper.py | Today at 8:10 PM |
| slider_control.py | Today at 8:10 PM |
| teleop_keyboard.py | Today at 8:10 PM |

## 2、Teleop 键盘控制节点存在物理安全风险

teleop_keyboard.py 使用 spin_until_future_complete 等待服务调用完成，未设置超时或降级策略，底层服务卡住时，控制端也会一起“挂住”。

```
future = self.set_coords_client.call_async(request)
rclpy.spin_until_future_complete(self, future)
if future.result() is not None:
    pass
```

初始坐标获取失败时，直接返回 [-1, -1, -1, -1, -1, -1] 作为占位值，后续仍在此基础上做增量移动，缺乏“检测异常 → 强制停机 / 重新标定”的逻辑。

```
else:
    self.get_logger().error('Failed to get coordinates')
    return [[-1, -1, -1, -1, -1, -1], self.speed, self.model]
```

键盘映射中没有预留明确的紧急停止（E-stop）按键，异常情况下只能依赖外部断电或手动关闭节点，响应速度不足。

```
while rclpy.ok():
    try:
        with Raw(sys.stdin):
            key = sys.stdin.read(1)
        if key == "q":
            break
        elif key in ["w", "W"]:
```

**这些问题不是传统意义上的“漏洞利用”，但会在异常场景下放大机械臂的物理风险，需要在后续版本中通过增加超时、错误状态检测和 E-stop 支持来改进。**

# 03

## 当前主要问题

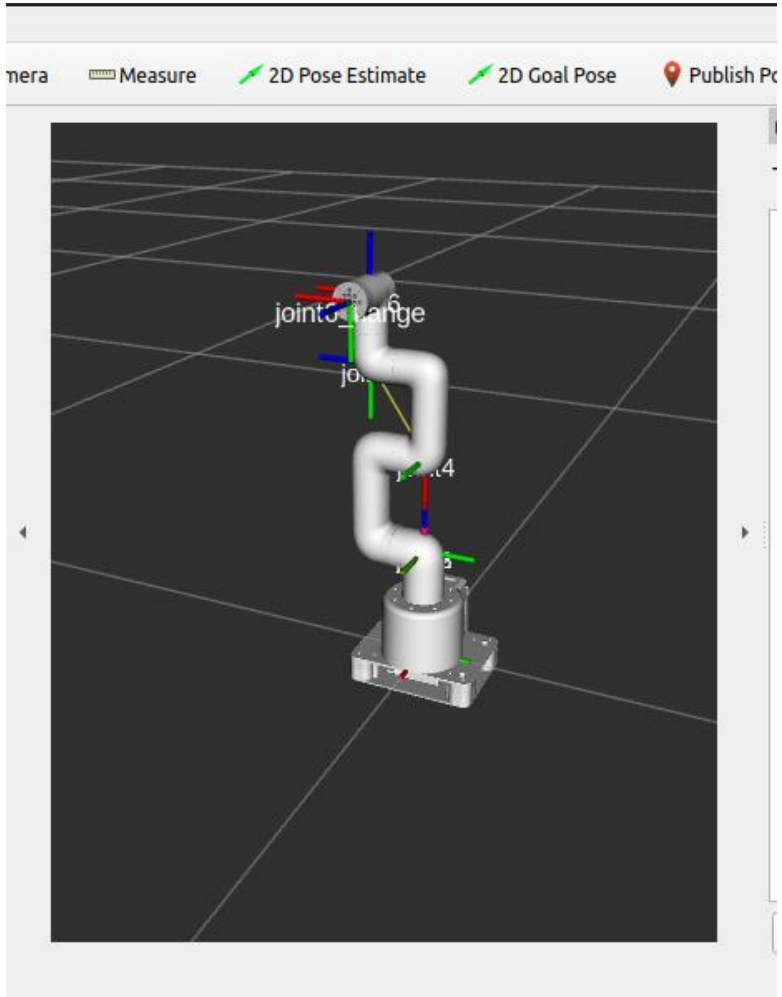
## 当前我们的面临的问题主要有以下四点

1. **硬件资源紧张**，我们需要与其他研究小组共用两台模型机械臂，很多思路和代码的验证不够及时。为此我们在积极与其他同学协调使用时间，保证两方都能顺利实验。

2. 当前用于ROS2和python漏洞扫描的工具远不如C、C++等代码审计工具**完善**，很难直接从代码发现不良或缺陷。

3. 环境搭建**依赖工具部分过时**，导致在setup阶段需要花费较长时间。为此我们在很多地方（包括机械臂模拟、MULVAL等）采用或尝试了虚拟机，并重新整理requirments.txt文件，使用虚拟环境开发。

4. **XSB直接输出的推理过程非常繁琐且有大量冗余信息**，需要通过脚本整理才能输出直接且易读的攻击路径。



# 04

## 下阶段工作

## 12月-1月

**完全复现或按论文思路成功构建一个小型自动化攻击路径推理系统。**

基于 MulVAL 逻辑框架，以机械臂Mycobot 280 Arduino为目标场景，用 Datalog 编码其 ROS2 漏洞、网络配置等事实，构建 “漏洞利用→权限提升” 的推理规则，确保能自动推理出基础攻击路径，并测试小型场景的推理兼容性。

## 2月-3月

**使用大模型自动构建推理规则，替代人工作用**

基于机械臂漏洞描述（如 ROS2 权限绕过）与 Datalog 规则模板，投喂大模型生成符合 MulVAL 语法的推理规则；将生成规则导入 XSB 引擎，验证其推理结果与人工规则的一致性；迭代优化prompt，提升规则的语法合规性与场景适配性，替代人工编写规则的流程。

## 1月-2月

**在模型机械臂上验证自动化路径推理系统的检测效果，并尝试验证推理系统的输出是否可行，及其潜在效果。**

搭建模型机械臂的 ROS2 运行环境，将推理系统接入机械臂网络；模拟未加密通信等漏洞，执行系统输出的攻击路径，验证是否能触发非授权关节控制；统计检测准确率，记录路径执行的实际效果，分析推理结果对应的安全风险。

# 环境配置与参考文献

## 环境配置

Ubuntu 20.04.6 LTS
python 3.10.7
ROS2 Jazzy
XSB 5.0

## AI使用声明

| | |
|---|---|
| Cursor | for understanding MulVAL |
| Chat-GPT | for the environment setup |
| Github CoPilot | for generating some repeated code |

## 参考论文

Ou, X., Govindavajhala, S., & Appel, A. W. (2005). MulVAL: A Logic-based Network Security Analyzer. In 14th USENIX Security Symposium (pp. 113–128). USENIX Association. [https://www.usenix.org/conference/14th-usenix-security-symposium/mulval-logic-based-network-security-analyzer]

# Thanks

## 恳请各位老师批评指正

T H A N K S